

BAB V

IMPLEMENTASI DAN PENGUJIAN

5.1 Implementasi Perancangan

5.1.1 Implementasi Model

Tahap ini dilakukan implementasi model dengan proses *import library* yang akan digunakan untuk *preprocessing*, implementasi model *Random Forest*, pengujian menggunakan metrik evaluasi, serta komponen pendukung lainnya.

Gambar 5.1 menunjukkan *source code* yang digunakan dalam proses tersebut.

```
!pip install micromlgen

# Library
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Preprocessing data
from sklearn.model_selection import train_test_split

# Random Forest
from sklearn.ensemble import RandomForestClassifier

# Pengujian Metrik
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, roc_auc_score, roc_curve

# Micromlgen
import micromlgen

# Google drive
from google.colab import drive
```

Gambar 5.1 Implementasi *Import Library*

Tahap berikutnya melakukan *preprocessing* untuk menyiapkan dataset sebelum digunakan dalam proses pelatihan model tersebut. Tahap *preprocessing*, diawali dengan pemeriksaan *missing data*, yang ditunjukkan pada Gambar 5.2.

```
# 1. Periksa missing value pada dataset irigasi
missing_values = data_irigation.isnull().sum()

# Tampilkan jumlah indikasi missing value pada fitur data
print("Jumlah sampel yang terindikasi missing value\npada fitur data:\n")
print(missing_values.to_frame(name="Jumlah missing")
      .rename_axis("Fitur Data")
      .reset_index()
      .to_string(index=False))
```

Gambar 5.2 Implementasi *Check Missing Value*

Kemudian, data tersebut dipisahkan menjadi fitur (x), dan target (y), di mana fitur mencakup kelembaban tanah, suhu udara, dan kelembaban udara, dan target adalah status pompa. Kemudian data dibagi menjadi 80% untuk *training* data dan 20% untuk *testing* data. Gambar 5.3 menunjukkan *source code* yang digunakan dalam proses tersebut.

```
# Pembagian Fitur dan Target
X = data_irigation[['Soil Moisture', 'Temperature', 'Air Humidity']]
y = data_irigation['Pump Data']

# Pembagian Data Menjadi Training dan Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8,
                                                    test_size=0.2, random_state=42)
```

Gambar 5.3 Implementasi Pemisahan Fitur dan Pembagian Data

Tahap selanjutnya, *training* data yang dihasilkan akan dilakukan pengecekan *outlier* menggunakan *boxplot* untuk visualisasi, sementara metode *Interquartile Range (IQR)* untuk validasi. Gambar 5.4 dan Gambar 5.5 menunjukkan *source code* yang digunakan dalam proses tersebut.

```
# 3. Buat boxplot untuk mendeteksi outlier secara visual
fig, axes = plt.subplots(1, 3, figsize=(10,4))

# Boxplot untuk fitur Soil Moisture
sns.boxplot(data=X_train, y='Soil Moisture', ax=axes[0], color="skyblue")
axes[0].set_title('Soil Moisture Outliers')

# Boxplot untuk fitur Temperature
sns.boxplot(data=X_train, y='Temperature', ax=axes[1], color="orange")
axes[1].set_title('Temperature Outliers')

# Boxplot untuk fitur Soil Moisture
sns.boxplot(data=X_train, y='Air Humidity', ax=axes[2], color="green")
axes[2].set_title('Air Humidity Outliers')

plt.tight_layout()
plt.show()
```

Gambar 5.4 Implementasi Visualisasi *Boxplot* Untuk Deteksi *Outlier*

```
# 3.1 Hitung IQR untuk mendeteksi outlier
Q1 = X_train.quantile(0.25)
Q3 = X_train.quantile(0.75)
IQR = Q3 - Q1

# Menghitung jumlah outlier per fitur
outliers = ((X_train < (Q1 - 1.5 * IQR)) | (X_train > (Q3 + 1.5 * IQR))).sum()

# Buat hasil outlier dalam bentuk tabel
outliers_table = pd.DataFrame({'Fitur': outliers.index, 'Jumlah Outlier' :
outliers.values})

outliers_table = outliers_table.reset_index(drop=True)

print(outliers_table.to_string(index=False))
```

Gambar 5.5 Implementasi Metode *IQR* Untuk Deteksi *Outlier*

Selanjutnya, dilakukan analisis keseimbangan kelas dengan membuat distribusi *plot*, serta tabel informasi yang terdiri dari fitur, jumlah data, dan persentase distribusi kelas, untuk mengidentifikasi ketidakseimbangan kelas berdasarkan selisih antara kelas mayoritas dan minoritas. Gambar 5.6 dan Gambar 5.7 menunjukkan *source code* yang digunakan untuk mengidentifikasi ketidakseimbangan kelas.

```

# 4. Lakukan pemeriksaan keseimbangan data

# Hitung jumlah dan persentase
class_distribution = y_train.value_counts()
class_percentages = (class_distribution / len(y_train)) * 100

# Buat laporan dalam bentuk tabel
class_distribution_table = pd.DataFrame({
    "Pump Data": class_distribution.index,
    "Jumlah": class_distribution.values,
    "Persentase %": class_percentages.values
})

# Rapikan angka desimal
class_distribution_table["Persentase %"] = class_distribution_table["Persentase
%"].round(2)

# Urutkan kelas
class_distribution_table = class_distribution_table.sort_values(by="Pump Data")

class_distribution_table = class_distribution_table.reset_index(drop=True)

print(class_distribution_table.to_string(index=False))

```

Gambar 5.6 Implementasi Analisis Keseimbangan Kelas

```

# Tampilkan visualisasi distribusi kelas melalui plot
plt.figure(figsize=(5, 5))
sns.barplot(x=class_distribution.index, y=class_distribution.values, width=0.5)
plt.title("Distribusi Kelas Target (Pump Data)", fontsize=12)
plt.xlabel("Kelas Target", fontsize=14)
plt.ylabel("Jumlah Sampel", fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

plt.show()

```

Gambar 5.7 Implementasi Visualisasi Plot Keseimbangan Kelas

Selanjutnya, dilakukan proses pengelompokan data ke dalam kategori berdasarkan kelembaban tanah, suhu udara, kelembaban udara, dan status udara. Gambar 5.8 menunjukkan *source code* fungsi pengelompokan data, sedangkan Gambar 5.9 menunjukkan *source code* yang digunakan untuk menampilkan hasil pengelompokan data.

```

# 5 Pengelompokan data
# Fungsi Pengelompokan Kelembaban Tanah
def categorize_soil_moisture(moisture):
    if moisture >= 727:
        return 'Sangat Kering'
    elif moisture >= 618:
        return 'Kering'
    elif moisture >= 508:
        return 'Normal'
    elif moisture >= 398:
        return 'Basah'
    else:
        return 'Sangat Basah'
# Fungsi Pengelompokan Suhu Udara
def categorize_temperature(temperature):
    if temperature <= 23:
        return 'Dingin'
    elif 24 <= temperature <= 30:
        return 'Normal'
    else:
        return 'Panas'
# Fungsi Pengelompokan Kelembaban Udara
def categorize_air_humidity(humidity):
    if humidity <= 55:
        return 'Kering'
    else:
        return 'Lembab'
# Fungsi Pengelompokan Pump Data
def categorize_pump_data(pump_data):
    if pump_data == 0:
        return 'Hidup'
    else:
        return 'Mati'

```

Gambar 5.8 Implementasi Fungsi Untuk Pengelompokan Data

```

soil_moisture = X_train['Soil Moisture']
soil_moisture_category = soil_moisture.apply(categorize_soil_moisture)

temperature = X_train['Temperature']
temperature_category = temperature.apply(categorize_temperature)

air_humidity = X_train['Air Humidity']
air_humidity_category = air_humidity.apply(categorize_air_humidity)

pump_data = y_train
pump_data_category = pump_data.apply(categorize_pump_data)

# Mensatukan soil_moisture dan soil_moisture_category
soil_moisture_df = pd.DataFrame({
    'Soil Moisture': soil_moisture,
    'Soil Moisture Category': soil_moisture_category
})
# Mensatukan temperature dan temperature_category
temperature_df = pd.DataFrame({
    'Temperature': temperature,
    'Temperature Category': temperature_category
})
# Mensatukan air_humidity dan air_humidity_category
air_humidity_df = pd.DataFrame({
    'Air Humidity': air_humidity,
    'Air Humidity Category': air_humidity_category
})
# Mensatukan pump_data dan pump_data_category
pump_data_df = pd.DataFrame({
    'Pump Data': pump_data,
    'Pump Data Category': pump_data_category
})
# Menggabungkan seluruh DataFrame menjadi satu untuk analisis
combined_df = pd.concat([soil_moisture_df, temperature_df, air_humidity_df,
pump_data_df], axis=1)
# Menampilkan DataFrame hasil gabungan
pd.set_option('display.max_rows', None) # Menampilkan semua baris
pd.set_option('display.max_columns', None) # Menampilkan semua kolom
combined_df.head(100) # Menampilkan 10 baris pertama

```

Gambar 5.9 Implementasi Lanjutan Proses Pengelompokan Data

Setelah tahap *preprocessing* data, dilakukan pengembangan model *Random Forest* dengan menentukan *hyperparameter*, seperti $n_estimators = 100$, *criterion* = 'gini', *max_features* = 'sqrt', *bootstrap* = *True*, serta variasi nilai *max_depth* yang dijelaskan sebelumnya. Pada proses tersebut, nilai *max_depth* diatur dalam rentang 3 hingga 7, di mana setiap variasi tersebut digunakan dalam proses pelatihan

metode. Gambar 5.4 menunjukkan *source code* yang digunakan dalam proses pengembangan dan pelatihan model tersebut.

```
# Random Forest Classifier

forest = RandomForestClassifier(n_estimators=100, max_features=
"sqrt", criterion="gini", bootstrap=True, max_depth=3,
random_state=42)

# Training model
forest.fit(X_train, y_train)
```

Gambar 5.10 Implementasi Model Random Forest

Setelah model dilatih, pengujian dilakukan untuk mengevaluasi kemampuan dalam memprediksi kebutuhan air pada tanaman. Evaluasi dilakukan dengan menggunakan metrik akurasi, *classification report*, *confusion matrix*, dan *AUC* (*Area Under Curve*). Gambar 5.11 hingga Gambar 5.13 menampilkan perhitungan dan visualisasi hasil pengujian.

```
# Pengujian Metrik
y_train_pred = forest.predict(X_train)
train_accuracy = accuracy_score(y_train, y_train_pred)

# Hitung akurasi testing
y_test_pred = forest.predict(X_test)
test_accuracy = accuracy_score(y_test, y_test_pred)

# Menampilkan nilai parameter max_depth
print(f"Max_depth: {forest.max_depth}")

# Tampilkan hasil evaluasi akurasi
print(f"Training Accuracy: {train_accuracy*100:.2f}%")
print(f"Test Accuracy: {test_accuracy*100:.2f}%")

# Tampilkan classification report

print("\nClassification Report:")
print(classification_report(y_test, y_test_pred, target_names= ["ON", "OFF"],
digits=4))
```

Gambar 5.11 Implementasi Pengukuran Metrik

```
# Hitung confusion matrix
cm = confusion_matrix(y_test, y_test_pred)

# Tampilkan Confusion Matrix
plt.figure(figsize=(3, 3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, xticklabels=
['ON', 'OFF'], yticklabels=['ON', 'OFF'])
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')
plt.show()
```

Gambar 5.12 Implementasi Confusion Matrix

```
# Prediksi probabilitas pada data test
# Ambil probabilitas kelas positif
y_test_proba = forest.predict_proba(X_test)[: , 1]

# Hitung AUC score sebagai metrik evaluasi
auc_score = roc_auc_score(y_test, y_test_proba)
print(f"AUC Score of Max Depth {forest.max_depth}: {auc_score}")

# Hitung nilai ROC curve (FPR & TPR)
fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)

# Visualisasi
plt.figure(figsize=(5, 3))
plt.plot(fpr, tpr, color='orange',
         label=f'ROC Curve (AUC = {auc_score*100:.2f})')
plt.plot([0, 1], [0, 1], color='black', linestyle='--', label='Random Forest
Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc='lower right')

plt.figtext(0.5, -0.15, f"Max Depth: {forest.max_depth}",
           ha="center", fontsize=10)

plt.show()
```

Gambar 5.13 Implementasi Pengukuran ROC-AUC

Terakhir, melakukan konversi model *Random Forest* yang telah dilatih dengan nilai *max_depth* terbaik ke dalam C++, sehingga menghasilkan kode (.h) yang akan diintegrasikan ke dalam sistem. Gambar 5.14 menunjukkan *source code* yang digunakan dalam proses konversi model *Random Forest*.


```

# Konversi Random Forest ke C++ menggunakan micromlgen
code = micromlgen.port(forest)

# Buat file
filename = f"random_forest_classifier_m_{forest.max_depth}.h"

# Simpan file
with open(filename, "w") as f:
    f.write(code)

print(f"Konversi berhasil\nFile:{filename}\nMax Depth:{forest.max_depth}")

```

Gambar 5.14 Implementasi Konversi *Random Forest* ke C++

5.1.2 Implementasi Perangkat Keras

Tahap implementasi perangkat keras dilakukan dengan proses perakitan komponen utama, yaitu sensor kelembaban tanah kapasitif, sensor DHT22, mikrokontroler ESP32, *relay*, dan pompa air. Sementara komponen pendukung yang terdiri dari baterai, panel surya, *solar charger controller*, *step-down*, dan kerangka menara sebagai tempat pemasangan perangkat.

Perakitan dimulai dengan menghubungkan baterai dan panel surya ke solar charger controller untuk mengatur proses pengisian dan distribusi daya. Output yang dihasilkan akan dihubungkan ke *step-down* pertama untuk menurunkan tegangan dari 12 V menjadi 7.5 V sebagai suplai daya ke ESP32 melalui breadboard. Sementara itu, *step-down* kedua digunakan untuk menurunkan tegangan 9.0 V sebagai suplai daya ke pompa air.

Berikut konfigurasi pin GPIO pada sensor dan aktuator yang ditunjukkan pada Tabel 5.1.

Tabel 5.1 Konfigurasi Pin GPIO pada Sensor dan Aktuator

No.	Hardware	Keterangan	<i>GPIO</i>	Voltase
1	<i>Capacitive Soil Moisture</i>	Membaca kondisi kelembaban tanah sebagai kadar air dalam tanah.	32	5 V
2	DHT22	Membaca suhu udara dan kelembaban udara.	22	3 V
3	<i>Relay 5V</i>	Mengontrol pompa air.	16	5 V

Hasil implementasi perangkat keras bagian internal pada sistem irigasi otomatis ditunjukkan pada Gambar 5.15 hingga Gambar 5.17.

**Gambar 5.15 Tampilan Rangkaian Kontrol Internal**



Gambar 5.16 Tampilan *Solar Charge Controller*



Gambar 5.17 Tampilan Pompa Air

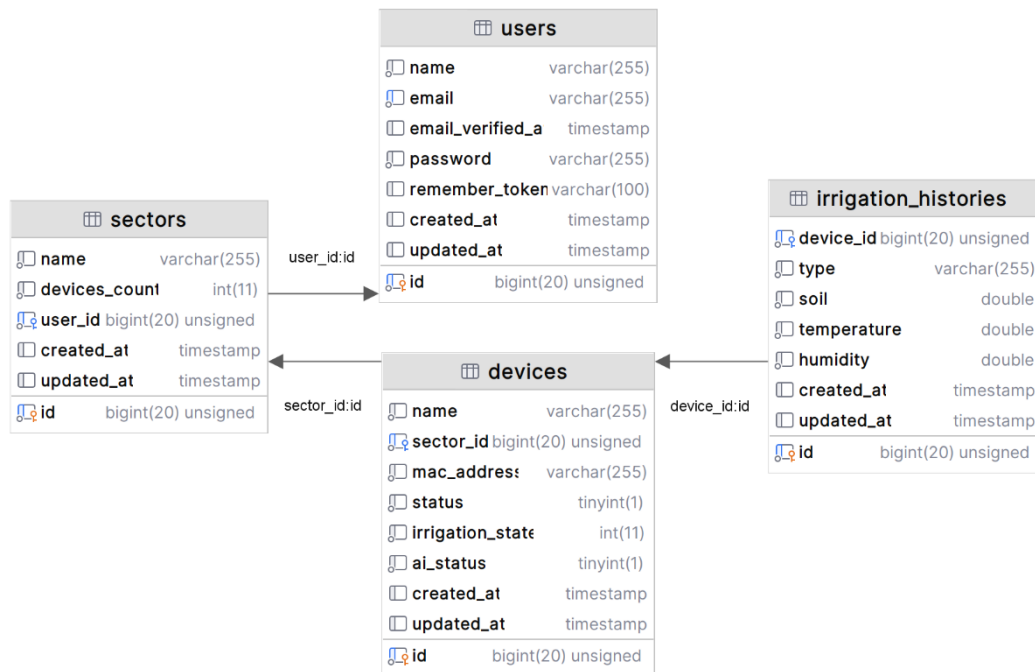
Hasil implementasi kerangka perangkat keras bagian *outdoor* pada sistem irigasi otomatis yang ditempatkan di lahan dapat ditunjukkan pada Gambar 5.18.



Gambar 5.18 Tampilan Kerangka Perangkat Keras Sistem Irigasi Otomatis

5.1.3 Implementasi Database

Tahap ini dilakukan implementasi *database* yang telah dirancang untuk menyimpan dan mengambil data melalui *server*, sehingga memungkinkan pengguna mengakses data dari jarak jauh. Hasil implementasi *database* yang menampilkan tabel dan relasi antar tabel dapat dilihat pada Gambar 5.19.



Gambar 5.19 Implementasi Desain *Database*

Tabel *users* digunakan untuk menyimpan informasi akun pengguna dalam proses *session* dan autentifikasi, sehingga pengguna dapat mengakses data yang berelasi serta layanan lainnya seperti *MQTT*.

Selanjutnya, tabel *sectors* digunakan untuk menyimpan informasi lokasi atau area penempatan perangkat. Dalam satu sektor terdapat beberapa perangkat keras yang dapat dikelompokkan berdasarkan area tersebut.

Selanjutnya, tabel *devices* digunakan untuk menyimpan informasi perangkat keras yang terdiri dari identitas perangkat, lokasi atau sektor yang dipilih, status, dan state perangkat.

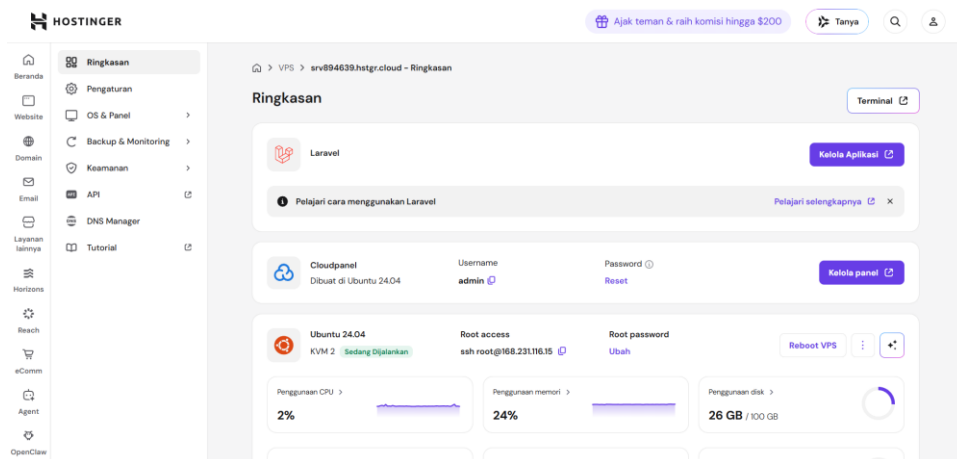
Terakhir, tabel *irrigation histories* digunakan untuk menyimpan riwayat penyiraman tanaman, baik secara otomatis maupun manual berdasarkan tipe

penyiraman, data sensor, dan tanggal riwayat tersebut. Satu perangkat dapat menghasilkan riwayat irigasi pada tanaman.

5.1.4 Implementasi Layanan Cloud Hosting

Tahap implementasi layanan cloud hosting dengan jenis *Virtual Private Server* (VPS) dilakukan untuk menempatkan API Laravel, MySQL *database server*, dan Mosquitto MQTT *broker* agar penyimpanan data serta layanan dapat diakses oleh perangkat dan aplikasi *client* melalui internet. Sistem ini menggunakan sistem operasi Linux Ubuntu 24.04 dengan spesifikasi yang sudah dijelaskan sebelumnya. Untuk konfigurasi sistem dilakukan melalui CloudPanel, yang menyediakan layanan database berupa MySQL.

Tampilan halaman utama yang berisi informasi dan pengaturan server pada Hostinger ditunjukkan pada Gambar 5.20.

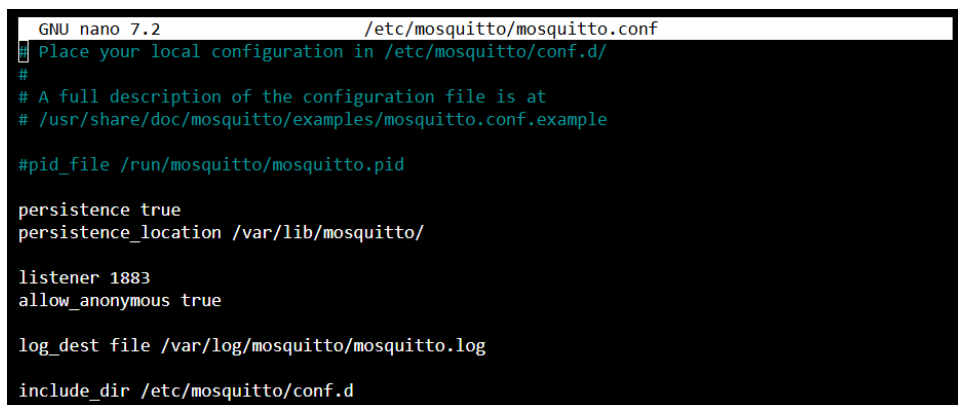


Gambar 5.20 Tampilan Halaman Ringkasan pada Admin Hostinger

Proses implementasi dimulai dengan menginstal Mosquitto MQTT *broker* yang berperan sebagai *server* protokol *machine-to-machine* untuk mengatur pengiriman data dari *publisher* dan penerimaan data dari *subscriber* secara *real-*

time. Broker ini memiliki keunggulan, yaitu *bandwidth* dan komputasi yang ringan, latensi rendah, serta konektivitas yang baik.

Instalasi Mosquitto dimulai dengan menjalankan perintah ‘`sudo apt install mosquitto mosquitto-clients -y`’. Selanjutnya, konfigurasi dilakukan dengan membuka berkas melalui perintah ‘`sudo nano /etc/mosquitto/mosquitto.conf`’ untuk memastikan parameter *listener* 1883 dan *allow_anonymous true* sudah diatur, sebagaimana ditunjukkan pada Gambar 5.21.



```
GNU nano 7.2 /etc/mosquitto/mosquitto.conf
# Place your local configuration in /etc/mosquitto/conf.d/
#
# A full description of the configuration file is at
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example
#pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto/

listener 1883
allow_anonymous true

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d
```

Gambar 5.21 Tampilan Konfigurasi Mosquitto MQTT Broker

Setelah konfigurasi selesai, layanan tersebut dijalankan menggunakan perintah ‘`sudo systemctl start mosquitto`’. Sementara itu, perintah ‘`sudo systemctl enable mosquitto`’ dilakukan agar layanan tersebut dapat berjalan secara otomatis saat VPS melakukan booting atau *restart*. Status aktivasi layanan Mosquitto MQTT broker dapat dilihat pada Gambar 5.22.

```
#####
###      Welcome to CloudPanel      ###
#####

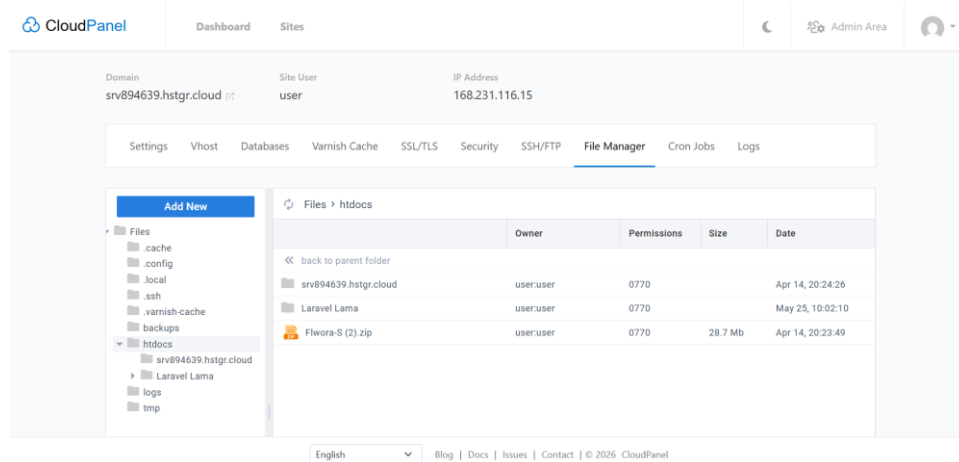
* Website:      https://www.cloudpanel.io
* Documentation: https://www.cloudpanel.io/docs/v2/
* Best Practices: https://www.cloudpanel.io/docs/v2/best-practices/
* CloudPanel:    https://168.231.116.15:8443
* CloudPanel CLI: clpctl

You have new mail.
Last login: Mon May 25 05:26:21 2026 from 169.254.0.1
root@srv894639:~# sudo nano /etc/mosquitto/mosquitto.conf^C
root@srv894639:~#
root@srv894639:~# sudo nano /etc/mosquitto/mosquitto.conf
root@srv894639:~# systemctl status mosquitto
● mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-05-22 03:42:04 UTC; 3 days ago
     Docs: man:mosquitto.conf(5)
           man:mosquitto(8).
   Main PID: 990 (mosquitto)
    Tasks: 1 (limit: 9483)
   Memory: 2.4M (peak: 2.9M)
      CPU: 2min 19.553s
   CGroup: /system.slice/mosquitto.service
           └─990 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

May 22 03:42:02 srv894639 systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...
May 22 03:42:04 srv894639 systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.
May 22 23:58:02 srv894639 systemd[1]: Reloading mosquitto.service - Mosquitto MQTT Broker...
May 22 23:58:02 srv894639 systemd[1]: Reloaded mosquitto.service - Mosquitto MQTT Broker.
May 23 23:58:01 srv894639 systemd[1]: Reloading mosquitto.service - Mosquitto MQTT Broker...
May 23 23:58:01 srv894639 systemd[1]: Reloaded mosquitto.service - Mosquitto MQTT Broker.
May 24 23:58:00 srv894639 systemd[1]: Reloading mosquitto.service - Mosquitto MQTT Broker...
May 24 23:58:00 srv894639 systemd[1]: Reloaded mosquitto.service - Mosquitto MQTT Broker.
root@srv894639:~#
```

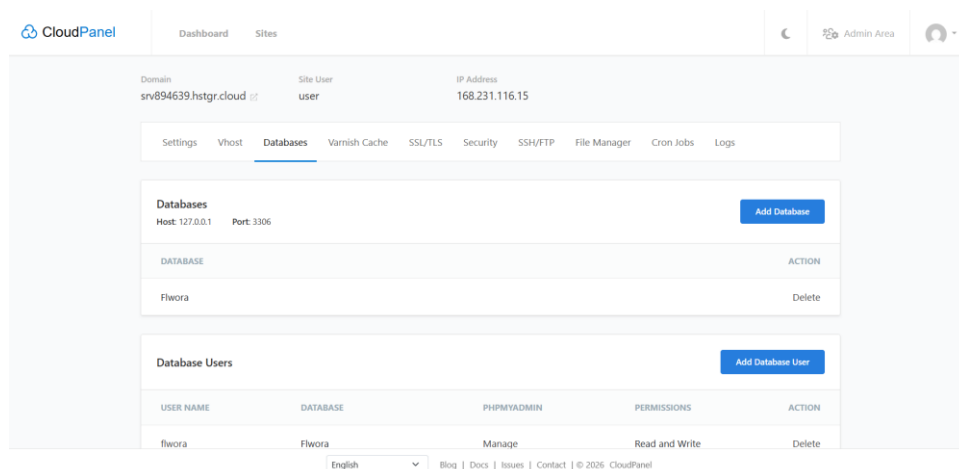
Gambar 5.22 Tampilan Status Mosquitto MQTT Broker

Selanjutnya, implementasi Laravel dilakukan dengan mengunggah berkas ke *server* melalui menu *File Manager* pada CloudPanel dalam format *.zip*, kemudian berkas tersebut diekstrak ke direktori `'files/htdocs/srv894639.hstgr.cloud'` yang dapat dilihat pada Gambar 5.23.



Gambar 5.23 Tampilan File Manager berisi Laravel pada CloudPanel

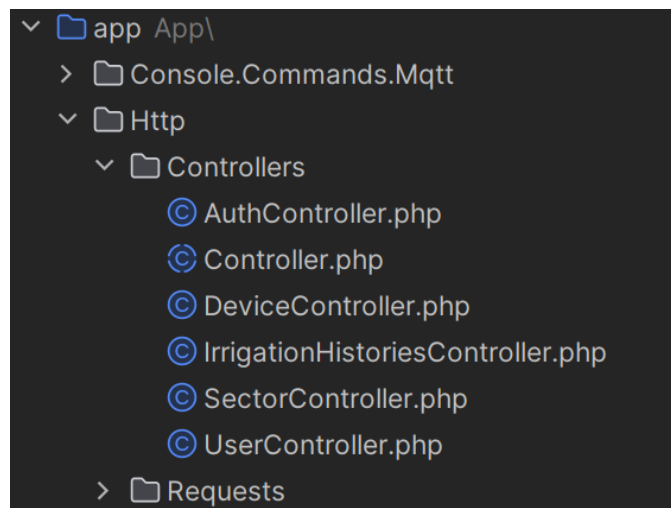
Setelah implementasi Laravel selesai, tahap selanjutnya membuat dan mengonfigurasi *database* dengan memasukkan nama *database*, *username*, dan *password*. Hasil *database* yang sudah ditambahkan dapat dilihat pada Gambar 5.24.



Gambar 5.24 Tampilan Konfigurasi Database MySQL pada CloudPanel

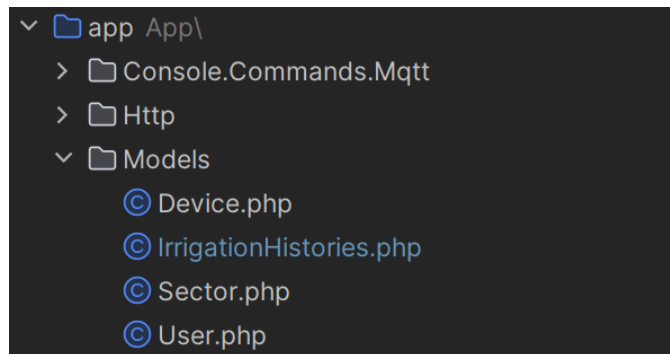
Pengembangan Laravel dilakukan dengan membuat komponen utama yang terdiri dari *model*, *controller*, *migration*, *console command*, dan *route*. *Controller* berperan sebagai unit kendali yang memproses permintaan pengguna dan mengirim data dalam bentuk *response* berupa *status code* maupun *status code* yang disertai data dalam format JSON. *Controller* yang dibuat terdiri dari *Auth Controller* untuk

proses autentikasi seperti login dan register. *User Controller* untuk mengubah *profile* dan *password* pengguna. *Sector Controller* digunakan untuk menyimpan, mengupdate, dan menghapus sektor atau lokasi lahan. *Device Controller* digunakan untuk menyimpan, mengupdate seperti perubahan nama dan pemindahan sektor, serta menghapus perangkat keras. Sementara itu, *Irrigation Histories Controller* digunakan untuk menampilkan riwayat irigasi yang dapat difilter berdasarkan *device id* dan tanggal. File kelas controller yang sudah dibuat ditunjukkan pada Gambar 5.25.

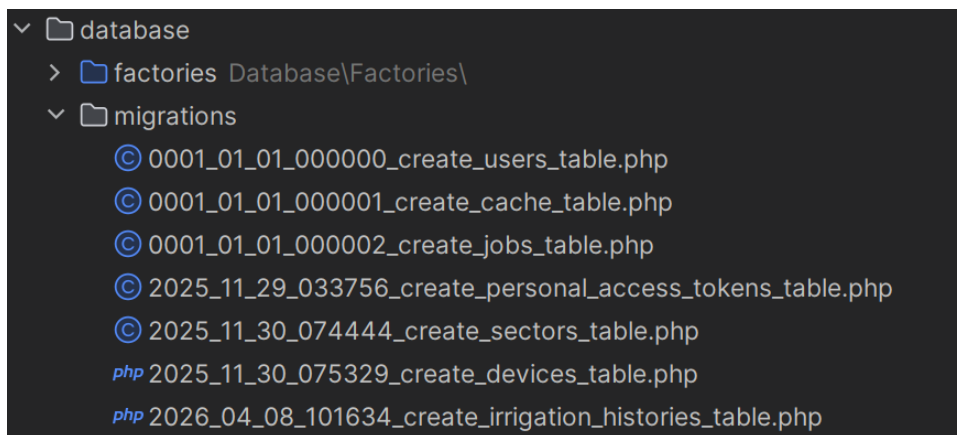


Gambar 5.25 File Kelas Controller pada Laravel

Model berperan sebagai representasi dari tabel dalam database, yang dibuat terdiri dari *User*, *Sector*, *Device*, dan *Irrigation Histories*. Sementara itu, *migration* digunakan untuk membuat struktur *database* pada setiap tabel. Kemudian, proses pembuatan tabel dilakukan dengan perintah ‘php artisan migrate’. File *model* dan *migration* yang telah dibuat dapat dilihat pada Gambar 5.26 dan Gambar 5.27.

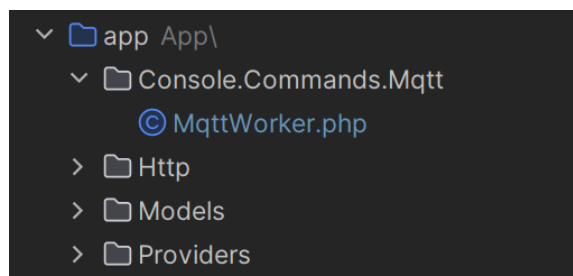


Gambar 5.26 File Kelas Model pada Laravel



Gambar 5.27 File Migration pada Laravel

Console command digunakan untuk menjalankan proses tertentu dengan perintah kustom. Salah satu *command* yang dibuat untuk menerima data dari MQTT *broker* secara real-time dan menyimpan data ke dalam *database*. File *console command* yang telah dibuat dapat dilihat pada Gambar 5.28.



Gambar 5.28 File Kelas Console Command MqttWorker

Selanjutnya, membuat file konfigurasi agar command dapat berjalan secara otomatis saat VPS melakukan booting atau *restart*. File tersebut dibuat menggunakan perintah ‘`sudo vim /etc/supervisor/conf.d/mqtt-subscriber-worker.conf`’, kemudian mengisi sesuai yang ditunjukkan pada Gambar 5.29.

```
program:mqtt-subscriber-worker
process_name=%(program_name)s_%(process_num)02d
directory=/home/user/htdocs/srv894639.hstgr.cloud
command=/usr/bin/php artisan app:mqtt-worker
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=user
numprocs=1
redirect_stderr=true
stdout_logfile=/home/user/logs/mqtt.log
stopwaitsecs=3600
~
~
~
~
~
```

Gambar 5.29 File Konfigurasi Supervisor MqttWorker

Setelah konfigurasi selesai, proses *console command* dijalankan menggunakan perintah ‘`sudo supervisorctl start "mqtt-subscriber-worker:*"`’.

Sementara itu, route dibuat yang terdiri dari autentikasi seperti *login* dan *register* menggunakan metode POST. Route user menggunakan metode PATCH untuk mengubah *profile* maupun *password*. Route sektor dan perangkat menggunakan metode POST, PATCH, dan DELETE, sedangkan route untuk menampilkan riwayat irigasi tanaman menggunakan metode GET.

5.2 Pengujian

5.2.1 Pengujian Model

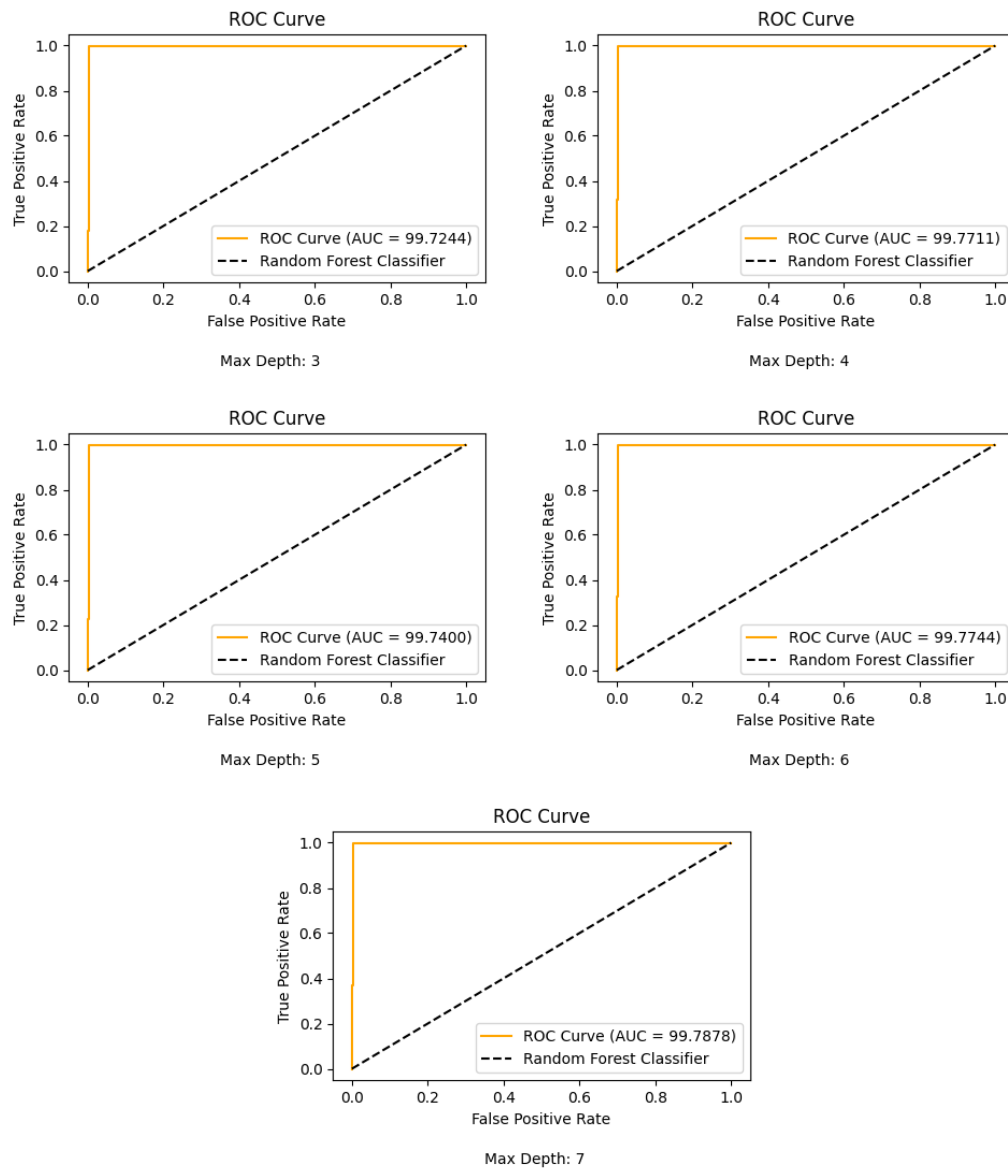
Pengujian ini dilakukan dengan menganalisis pengaruh *max_depth* terhadap kemampuan dalam memprediksi air pada tanaman, serta kompleksitas yang

berdampak pengguna memori. Nilai *max_depth* akan diuji dalam rentang 3 hingga 7 secara terpisah untuk melihat perbedaan performa dan kebutuhan komputasi. Memilih parameter terbaik didasarkan pada performa yang diukur melalui metrik akurasi, *precision*, *recall*, *f1-score*, *confusion matrix*, dan *AUC* (*Area Under Curve*). Akan tetapi, efisiensi pengguna memori menjadi pertimbangan dalam pemilihan parameter tersebut, dikarenakan keterbatasan memori pada mikrokontroler.

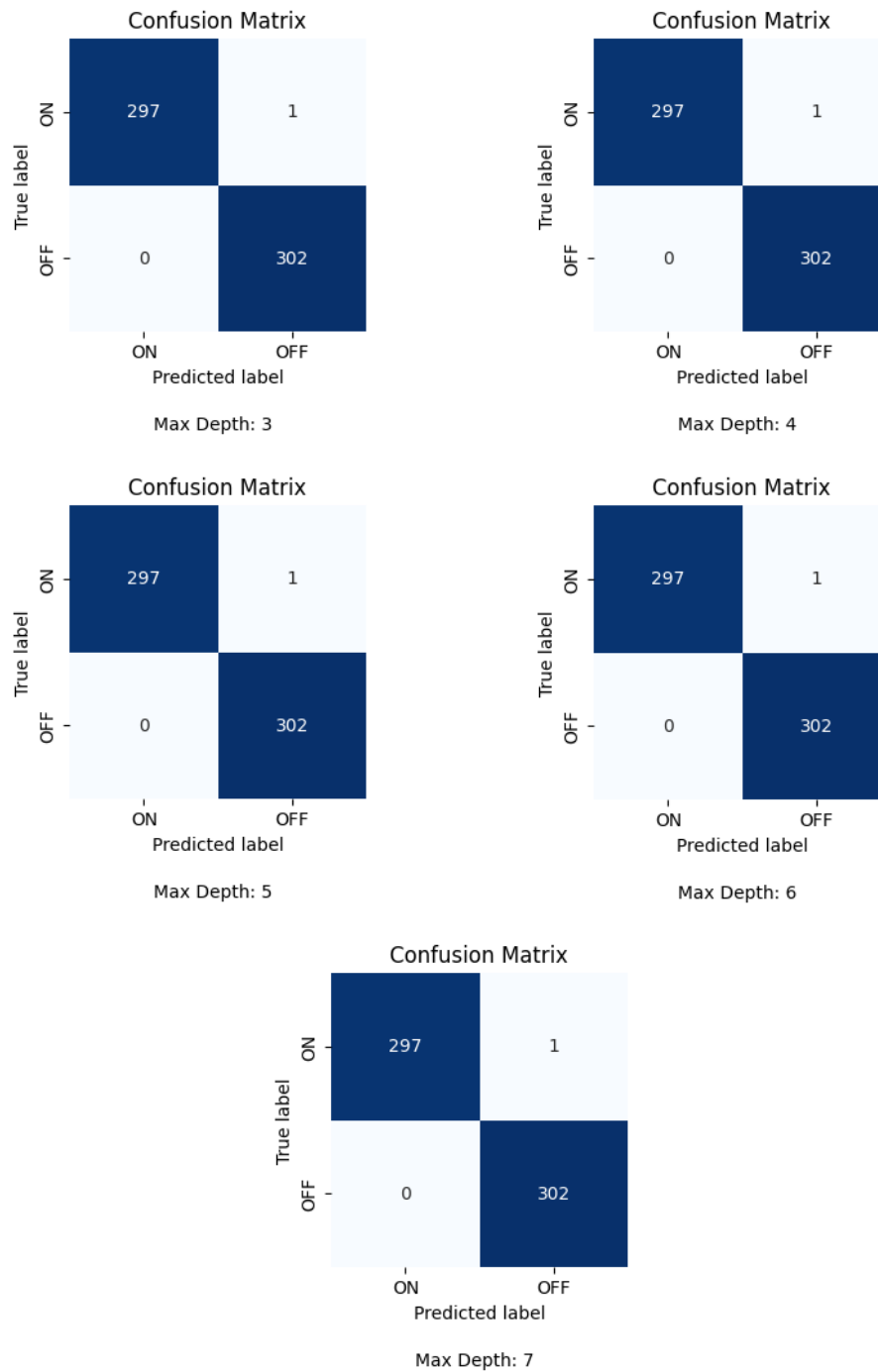
Tabel 5.2 Analisis Performa *Random Forest* pada Variasi Max Depth

<i>Max Depth</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>	<i>AUC</i>
3	99.83	99.83	99.83	99.83	99.72
4	99.83	99.83	99.83	99.83	99.77
5	99.83	99.83	99.83	99.83	99.74
6	99.83	99.83	99.83	99.83	99.77
7	99.83	99.83	99.83	99.83	99.79

Tabel 5.2 menunjukkan parameter *max_depth* yang bervariasi tidak memberikan peningkatan signifikan terhadap performa model dalam memprediksi kebutuhan air pada tanaman pada data pengujian, karena menghasilkan nilai metrik seperti akurasi, *precision*, *recall*, dan *f1-score* secara konsisten sebesar 99.83%. Meskipun nilai *AUC* (*Area Under Curve*) mengalami sedikit peningkatan, yang tidak memberikan dampak terhadap kinerja model. Sementara itu, dalam data pelatihan, *max_depth* = 3 hingga 5 menghasilkan akurasi sebesar 99.96%, sedangkan *max_depth* = 6 dan 7 menghasilkan akurasi sebesar 100%.



Gambar 5.30 Kurva *ROC-AUC* pada Variasi *Max Depth*



Gambar 5.31 Confusion Matrix pada Variasi Max Depth

Gambar 5.31 menunjukkan setiap *max_depth* yang meningkat menghasilkan klasifikasi yang sama, dengan *TP* (*True Positive*) sebesar 297, *False Negative* (*FN*) sebesar 1, *FP* (*False Positive*) sebesar 0, dan *TN* (*True Negative*) sebesar 302.